Programación orientada a objetos y modelado de software

Breve descripción:

Este componente formativo desarrolla capacidades para analizar, diseñar e implementar sistemas de software orientados a objetos mediante UML, favoreciendo la construcción de soluciones modulares, escalables y seguras, preparadas para su implementación en entornos de computación en la nube.

Septiembre 2026

Tabla de contenido

Introducción 4

1. Fundamentos de Programación Orientada a Objetos (POO) 7

1.1. Características de la programación orientada a objetos 7

1.2. Clases y objetos 10

1.3. Atributos y métodos 12

1.4. Entradas y salidas en aplicaciones orientadas a objetos 14

1.5. Interacción entre objetos 17

2. Modelado y diseño de software 20

2.1. Fundamentos del modelado UML 20

2.2. Elementos básicos del UML 22

2.3. Diagramas de clases 23

2.4. Relaciones entre clases 26

2.5. Herramientas CASE para el modelado de software 28

3. Estructura e implementación de clases 30

3.1. Implementación de una clase 30

3.2. Especificadores de acceso 33

3.3. Constructores y administración de memoria en Java 35

4. Herencia en la programación orientada a objetos 38

4.1. Principios y beneficios de la herencia 38

4.2. Herencia simple y múltiple 40

4.3. Sobrescritura de métodos (override) 42

4.4. Clases y métodos abstractos 44

4.5. Constructores en la herencia 45

5. Polimorfismo y reutilización de código 48

5.1. Principios y beneficios del polimorfismo 48

5.2. Sobrecarga de métodos 51

5.3. Sobrescritura y polimorfismo 53

5.4. Aplicación del polimorfismo mediante herencia 57

6. Desarrollo y depuración de aplicaciones 60

6.1. Diseño e implementación de algoritmos 60

6.2. Depuración de programas 63

6.3. Fallas de sintaxis y fallas de lógica 64

6.4. Pruebas y corrección de errores 67

Síntesis 69

Glosario 70

Referencias bibliográficas 72

Créditos 73

Introducción

La programación orientada a objetos y el modelado de software constituyen fundamentos esenciales para el desarrollo de aplicaciones que respondan a los requerimientos actuales de calidad, escalabilidad y mantenibilidad. Este enfoque permite representar situaciones del mundo real mediante clases y objetos, favoreciendo la organización del código, la reutilización de componentes y la construcción de soluciones de software estructuradas.

En este componente se abordan los principios de la programación orientada a objetos, como el encapsulamiento, la herencia, el polimorfismo y la abstracción, así como el uso del Lenguaje Unificado de Modelado (UML) para el diseño y la representación de sistemas de software. Estos conocimientos facilitan el análisis, diseño e implementación de aplicaciones mediante modelos que apoyan la comunicación entre los integrantes del equipo de desarrollo y orientan el proceso de construcción del software.

El desarrollo de estas competencias proporciona las bases para la implementación de aplicaciones orientadas a objetos y fortalece la preparación del aprendiz para abordar componentes posteriores relacionados con el desarrollo web, la construcción de API, las prácticas DevOps y la implementación de soluciones en entornos de computación en la nube.

Video 1. Programación orientada a objetos y modelado de software



Enlace de reproducción del video

Transcripción del video. Programación orientada a objetos y modelado de software
El desarrollo de software ha evolucionado para responder a problemas cada vez más complejos. En sus inicios, los programas estaban compuestos por largas secuencias de instrucciones que resultaban difíciles de mantener y ampliar. Con el crecimiento de las aplicaciones surgió la necesidad de organizar mejor el código y facilitar su reutilización. Como respuesta a este desafío nació la programación orientada a objetos, un paradigma que transformó la manera de diseñar y construir soluciones informáticas, al representar los elementos del mundo real mediante clases y objetos. Este enfoque permitió desarrollar aplicaciones más organizadas, flexibles y fáciles de mantener.

Transcripción del video. Programación orientada a objetos y modelado de software
Con el tiempo, también se incorporaron herramientas de modelado, como el Lenguaje Unificado de Modelado (UML), que facilitan la planificación de la estructura del software antes de su implementación, así como lenguajes de programación, entre ellos Java, que permiten materializar estos diseños mediante principios como la herencia, el polimorfismo y el encapsulamiento. Sin embargo, el desarrollo de una aplicación no concluye con la escritura del código. La implementación de algoritmos, la identificación y corrección de errores, así como la realización de pruebas, constituyen etapas fundamentales para garantizar el funcionamiento adecuado y la calidad del software. Comprender estos fundamentos permite desarrollar aplicaciones más organizadas, mantenibles y adaptables a diferentes necesidades, fortaleciendo las competencias para participar en la construcción de soluciones de software de calidad.

1. Fundamentos de Programación Orientada a Objetos (POO)

La Programación Orientada a Objetos (POO) es un paradigma de programación que organiza el desarrollo de software mediante objetos, los cuales representan entidades con características (atributos) y comportamientos (métodos). Este enfoque facilita la representación de situaciones del mundo real, favorece la reutilización del código, mejora la organización de las aplicaciones y simplifica su mantenimiento y evolución.

A diferencia de la programación procedimental, que centra el desarrollo en funciones y procedimientos, la POO estructura el software alrededor de objetos que interactúan entre sí para cumplir un propósito específico. Este paradigma constituye una base para el desarrollo de aplicaciones modernas, ya que promueve soluciones modulares, escalables y mantenibles, características necesarias para la implementación de software en entornos de computación en la nube.

1.1. Características de la programación orientada a objetos

La programación orientada a objetos se fundamenta en cuatro principios que orientan el diseño y desarrollo de aplicaciones organizadas, reutilizables y fáciles de mantener. Estos principios permiten representar entidades del mundo real mediante clases y objetos, favoreciendo la modularidad, la escalabilidad y la calidad del software.

a. **Abstracción**

- **Definición:** es el proceso mediante el cual se identifican las características esenciales de un objeto, omitiendo aquellos detalles que

no son relevantes para el problema que se desea resolver. De esta manera, el desarrollador se concentra en las funcionalidades que ofrece un objeto más que en su implementación interna.

- Aplicación en el desarrollo de software: la abstracción facilita el uso de componentes, bibliotecas y servicios sin necesidad de conocer todos los detalles de su funcionamiento interno, lo que simplifica el desarrollo y favorece la reutilización del software.
- Ejemplo: al conducir un automóvil, el conductor utiliza el volante, los pedales y la palanca de cambios para operarlo, sin necesidad de conocer el funcionamiento interno del motor.

b. Encapsulamiento

- Definición: consiste en agrupar los datos (atributos) y las operaciones (métodos) que actúan sobre ellos dentro de una misma clase, controlando el acceso a la información mediante mecanismos de visibilidad. Esto protege la integridad de los datos y evita modificaciones no autorizadas.
- Aplicación en el desarrollo de software: el encapsulamiento contribuye a mantener la consistencia de la información, reducir errores y facilitar el mantenimiento de las aplicaciones.
- Ejemplo: una cápsula farmacéutica protege su contenido y permite acceder a él únicamente de la forma prevista, evitando alteraciones externas.

c. **Herencia**

- Definición: es el mecanismo mediante el cual una clase puede adquirir atributos y métodos de otra clase, permitiendo reutilizar funcionalidades comunes y especializar comportamientos cuando sea necesario.
- Aplicación en el desarrollo de software: la herencia favorece la reutilización del código, disminuye la duplicidad de funcionalidades y facilita la creación de jerarquías entre clases relacionadas.
- Ejemplo: un teléfono inteligente conserva funciones básicas de comunicación presentes en modelos anteriores, incorporando nuevas características como acceso a internet o aplicaciones móviles.

d. **Polimorfismo**

- Definición: es la capacidad de que una misma operación pueda ejecutarse de manera diferente según el tipo de objeto que la implemente. Esto permite utilizar una interfaz común para trabajar con objetos de distintas clases.
- Aplicación en el desarrollo de software: el polimorfismo incrementa la flexibilidad del código, facilita su extensión y favorece el desarrollo de aplicaciones adaptables a diferentes escenarios.
- Ejemplo: al seleccionar la opción Imprimir en una aplicación, el mismo comando puede enviar la información a una impresora láser, una impresora de tinta o un archivo PDF, dependiendo del dispositivo o destino configurado.

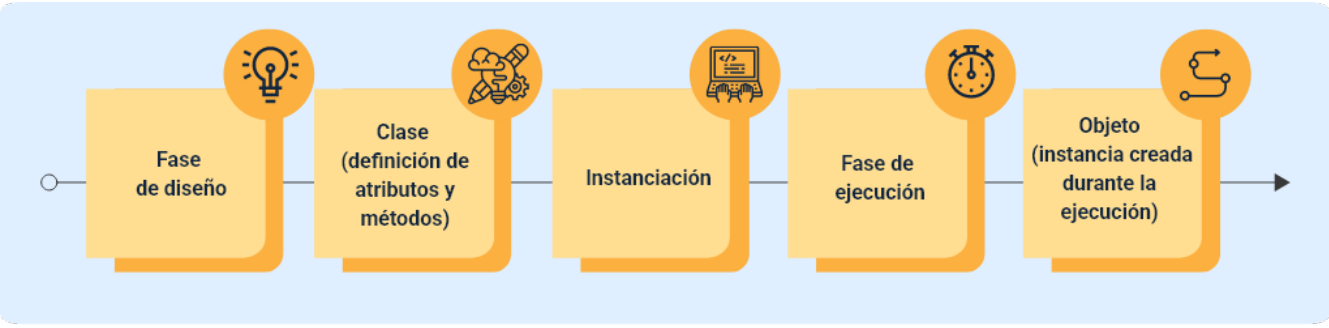
1.2. Clases y objetos

Las clases y los objetos constituyen los elementos fundamentales de la programación orientada a objetos. Aunque están estrechamente relacionados, representan conceptos diferentes. Una clase define la estructura y el comportamiento que compartirán un conjunto de objetos mediante atributos y métodos, mientras que un objeto corresponde a una instancia creada a partir de esa clase durante la ejecución del programa.

En otras palabras, la clase funciona como una plantilla que establece las características comunes de una entidad, mientras que los objetos representan las instancias que interactúan dentro de la aplicación. Cada objeto posee su propio estado, determinado por los valores de sus atributos, pero comparte la estructura y el comportamiento definidos por la clase.

La creación de un objeto es un proceso que inicia durante la etapa de diseño del software y finaliza cuando la aplicación se encuentra en ejecución. A continuación, se presenta de manera general esta secuencia.

Figura 1. Cronología del ciclo de diseño a ejecución



En la figura 1 se evidencia que, durante la fase de diseño, el desarrollador define la clase mediante el código fuente. Posteriormente, cuando la aplicación se ejecuta, el lenguaje de programación crea los objetos mediante el proceso de instanciación, asignando la memoria necesaria para que puedan almacenar información y ejecutar las operaciones definidas en la clase.

Comprender esta diferencia resulta fundamental para diseñar aplicaciones orientadas a objetos, ya que una misma clase puede utilizarse para crear múltiples objetos independientes que colaboran entre sí durante la ejecución del software.

Aunque ambos conceptos mantienen una relación directa, presentan diferencias en cuanto a su propósito, existencia y comportamiento durante la ejecución de una aplicación. La siguiente tabla resume las principales características que permiten distinguir una clase de un objeto:

Tabla 1. Comparativa entre clases y objetos

Criterio de comparación	Clase	Objeto (instancia)
Naturaleza	Define la estructura y el comportamiento de una entidad.	Es una instancia creada a partir de una clase.
Propósito	Sirve como plantilla para crear objetos.	Representa una entidad específica con valores propios para sus atributos.
Existencia	Se define durante el desarrollo del software mediante el código fuente.	Se crea durante la ejecución de la aplicación cuando es requerido.
Ubicación durante la ejecución	Su definición forma parte del programa y sirve como referencia para crear objetos.	Se almacena en memoria mientras permanece en uso por la aplicación.

Criterio de comparación	Clase	Objeto (instancia)
Cantidad	Una clase puede utilizarse para crear múltiples objetos.	Pueden existir numerosas instancias de una misma clase, cada una con información diferente.
Ciclo de vida	Permanece como parte de la estructura del software.	Existe mientras la aplicación requiere su utilización; posteriormente libera los recursos asociados.

Como ejemplo, una aplicación de gestión académica puede definir una clase denominada Aprendiz, la cual contiene atributos como nombre, documento y programa de formación, además de métodos para registrar información o consultar el estado del aprendiz. Durante la ejecución del sistema, cada persona registrada corresponde a un objeto diferente creado a partir de esa misma clase, conservando la misma estructura, pero almacenando información propia.

Esta capacidad de crear múltiples objetos a partir de una única definición favorece la reutilización del código, facilita el mantenimiento del software y permite desarrollar aplicaciones más organizadas, escalables y fáciles de evolucionar.

1.3. Atributos y métodos

Los objetos se caracterizan por dos elementos fundamentales: los atributos, que describen sus características o estado, y los métodos, que representan las acciones o comportamientos que pueden realizar. La combinación de estos elementos permite modelar entidades del mundo real y definir cómo interactúan dentro de una aplicación.

Atributos

- Los atributos corresponden a las propiedades o características que describen un objeto. Cada atributo almacena información que identifica el estado del objeto en un momento determinado. Estos pueden representar diferentes tipos de datos, como texto, números, fechas o valores lógicos.
- Por ejemplo, en una clase denominada Aprendiz, algunos atributos podrían ser nombre, documento, programa de formación y estado de matrícula. Cada objeto creado a partir de esta clase tendrá valores propios para dichos atributos.
- Con el fin de proteger la información y garantizar su integridad, los atributos generalmente se mantienen con acceso restringido. Su consulta o modificación se realiza mediante métodos definidos para tal propósito, aplicando el principio de encapsulamiento.

Métodos

- Los métodos representan las acciones o comportamientos que puede ejecutar un objeto. Mediante ellos es posible procesar información, modificar el estado de un objeto o interactuar con otros objetos dentro del sistema.
- Por ejemplo, una clase Aprendiz podría disponer de métodos como matricular(), actualizarDatos(), consultarEstado() o finalizarPrograma(), los cuales permiten realizar operaciones relacionadas con la información del aprendiz.

- Los métodos también pueden recibir parámetros, devolver resultados o ejecutar procesos internos necesarios para el funcionamiento de la aplicación.

A continuación, se presenta un ejemplo simplificado de la estructura de una clase, en la que se identifican sus atributos y métodos:

Clase: Aprendiz

Atributos

- documento: String

- nombre: String

- programa: String

- estadoMatricula: Boolean

Métodos

+ matricular()

+ actualizarDatos()

+ consultarEstado()

+ finalizarPrograma()

1.4. Entradas y salidas en aplicaciones orientadas a objetos

Las aplicaciones orientadas a objetos funcionan mediante la interacción entre diferentes objetos que reciben información, ejecutan operaciones y generan resultados.

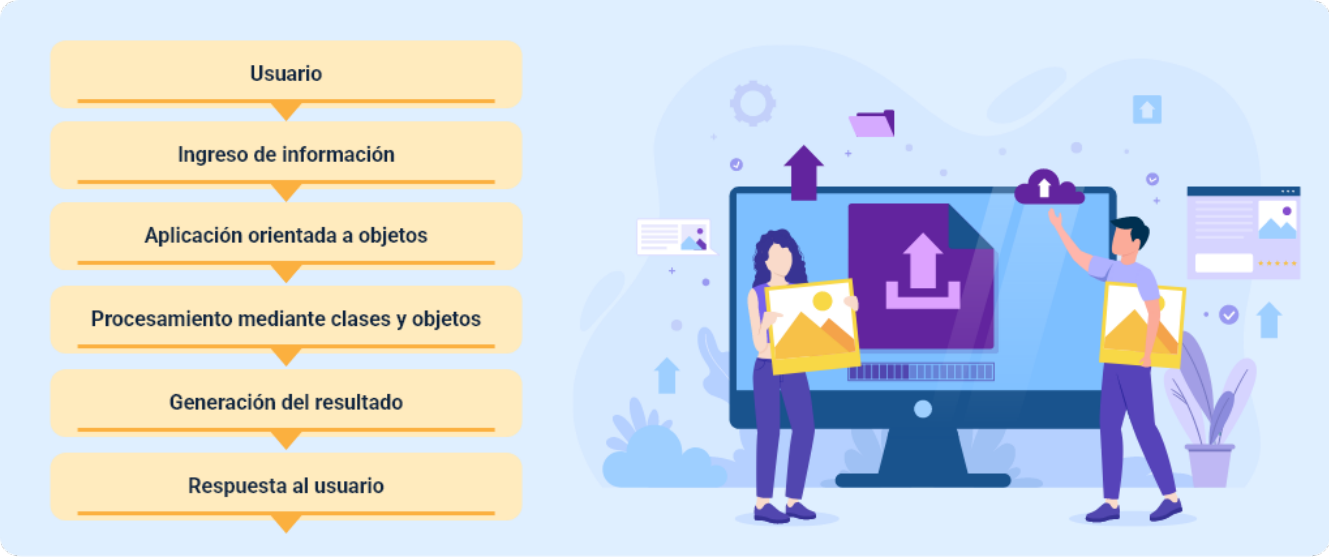
Este intercambio de datos constituye el flujo de entrada y salida de información dentro de una aplicación y permite que los objetos colaboren para cumplir las funcionalidades definidas por el sistema.

Las entradas corresponden a la información que recibe un objeto para ejecutar una determinada operación. Estos datos pueden provenir del usuario, de otro objeto, de una base de datos o de un servicio externo. Una vez recibida la información, el objeto procesa los datos mediante los métodos definidos en su clase.

Las salidas representan los resultados generados después del procesamiento. Dependiendo de la funcionalidad desarrollada, el resultado puede mostrarse al usuario, almacenarse en una base de datos, enviarse a otro componente del sistema o utilizarse como entrada para nuevos procesos.

A continuación, se presenta el flujo general de procesamiento de información en una aplicación orientada a objetos:

Figura 2. Flujo general de entrada y salida de información en una aplicación orientada a objetos



En este proceso se puede identificar cómo la información es recibida por la aplicación, procesada por los objetos que conforman la lógica del sistema y posteriormente transformada en una respuesta para el usuario o para otros componentes de la aplicación. Aunque en aplicaciones modernas este intercambio suele realizarse mediante protocolos de comunicación y formatos de intercambio de datos, el principio fundamental continúa siendo el mismo: recibir información, procesarla y generar un resultado.

Ejemplo de procesamiento de información:

Supóngase una aplicación para la gestión de aprendices del SENA.

Entrada: el usuario registra el nombre, el número de documento y el programa de formación.

Procesamiento: el objeto Aprendiz valida la información recibida y ejecuta el método registrar ().

Salida: la aplicación confirma el registro y almacena la información correspondiente.

Este flujo de trabajo demuestra cómo los objetos reciben información, ejecutan operaciones mediante sus métodos y generan resultados como parte del funcionamiento normal de una aplicación orientada a objetos.

Los procesos descritos anteriormente se apoyan en los principios fundamentales de la programación orientada a objetos, los cuales orientan el diseño de aplicaciones organizadas, reutilizables y fáciles de mantener. Estos principios son:

- **Abstracción:** permite representar únicamente las características esenciales de un objeto, ocultando los detalles innecesarios para el problema que se desea resolver.
- **Encapsulamiento:** protege la información contenida en los objetos mediante el control del acceso a sus atributos y métodos, favoreciendo la integridad de los datos.
- **Herencia:** posibilita crear nuevas clases a partir de otras existentes, reutilizando atributos y métodos comunes para facilitar el desarrollo y mantenimiento del software.
- **Polimorfismo:** permite que una misma operación tenga diferentes comportamientos según el objeto que la implemente, proporcionando mayor flexibilidad y extensibilidad a las aplicaciones.

La aplicación conjunta de estos principios favorece el desarrollo de software modular, facilita la reutilización del código y contribuye a construir aplicaciones escalables, mantenibles y adaptables a diferentes necesidades.

1.5. Interacción entre objetos

En una aplicación orientada a objetos, los objetos no funcionan de manera aislada. Para cumplir con los requerimientos del sistema, interactúan entre sí mediante el intercambio de información y la invocación de métodos. Esta colaboración permite distribuir las responsabilidades entre diferentes objetos, favoreciendo la organización, reutilización y mantenibilidad del software.

La interacción se produce cuando un objeto solicita a otro la ejecución de una operación específica. Para ello, envía un mensaje acompañado, si es necesario, de los datos requeridos para realizar la tarea. El objeto receptor procesa la solicitud mediante uno de sus métodos y devuelve el resultado correspondiente.

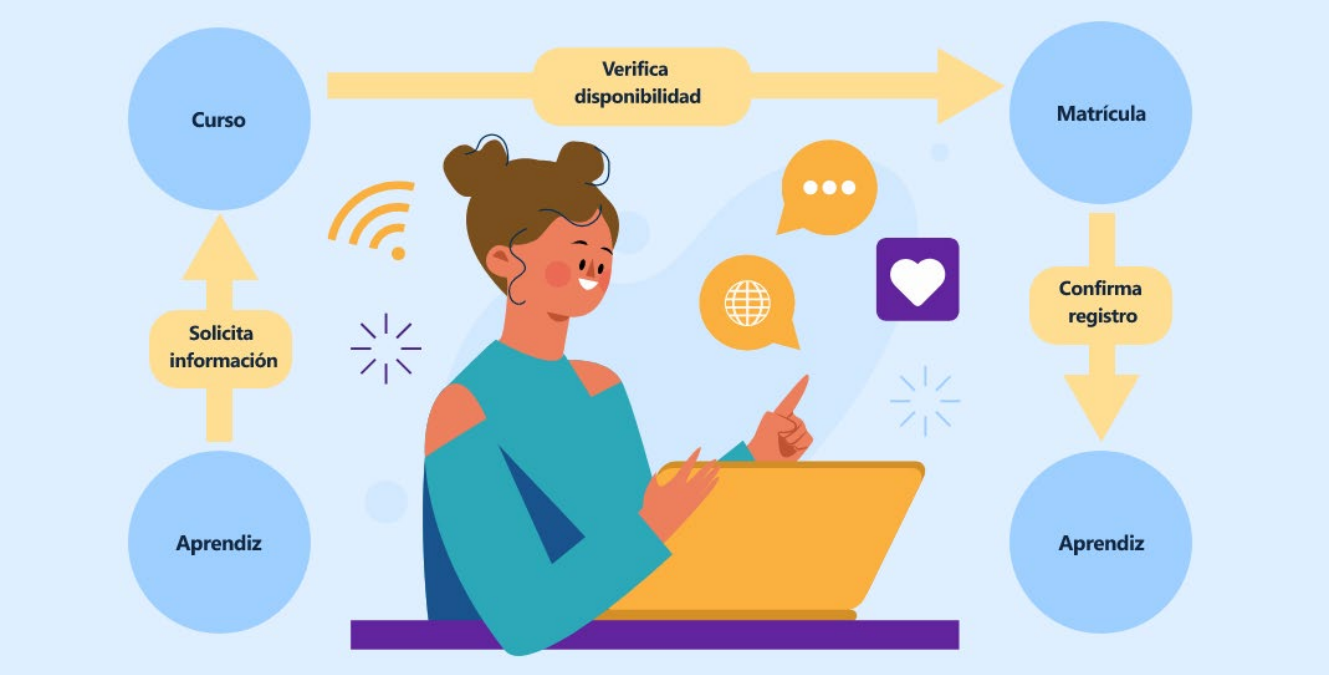
La comunicación entre objetos puede involucrar diferentes acciones, entre las que se destacan:

- **Invocación de métodos:** un objeto solicita a otro la ejecución de una operación determinada.
- **Intercambio de información:** los objetos envían o reciben datos necesarios para realizar un proceso.
- **Paso de parámetros:** un método recibe información suministrada por otro objeto para ejecutar una tarea específica.
- **Retorno de resultados:** una vez finalizado el procesamiento, el método puede devolver un valor o confirmar la ejecución de la operación.

Como ejemplo, en una aplicación de gestión académica, el objeto Aprendiz puede solicitar al objeto Curso la información correspondiente a un programa de formación. Posteriormente, el objeto Curso consulta los datos disponibles y devuelve la información solicitada, permitiendo que el objeto Aprendiz continúe con el proceso de matrícula o consulta.

A continuación, se presenta un ejemplo simplificado de la interacción entre objetos dentro de una aplicación orientada a objetos.

Figura 3. Interacción entre objetos en una aplicación orientada a objetos



La interacción entre objetos constituye uno de los principios fundamentales de la programación orientada a objetos, ya que permite desarrollar aplicaciones modulares, escalables y de fácil mantenimiento mediante la colaboración entre clases con responsabilidades claramente definidas.

2. Modelado y diseño de software

El modelado y diseño de software constituye una etapa fundamental del proceso de desarrollo, ya que permite definir la estructura, el comportamiento y la organización de una aplicación antes de iniciar su implementación. Durante esta fase se analizan los requerimientos del sistema, se identifican sus componentes y se establecen las relaciones e interacciones necesarias para satisfacer las necesidades del usuario.

El modelado facilita la representación gráfica de una solución mediante notaciones estandarizadas, lo que favorece la comprensión del sistema, la comunicación entre los integrantes del equipo de desarrollo y la identificación de posibles inconsistencias antes de la etapa de codificación. Asimismo, el diseño proporciona una guía para construir aplicaciones organizadas, mantenibles y escalables, reduciendo el riesgo de errores durante su desarrollo.

En este contexto, el Lenguaje Unificado de Modelado (UML) constituye uno de los estándares más utilizados para representar la estructura y el comportamiento de los sistemas orientados a objetos, mediante diagramas que apoyan las actividades de análisis, diseño, documentación y mantenimiento del software.

2.1. Fundamentos del modelado UML

El Lenguaje Unificado de Modelado (UML, por sus siglas en inglés Unified Modeling Language) es un estándar utilizado para representar de forma gráfica la estructura y el comportamiento de un sistema de software. Su propósito es facilitar el análisis, diseño, documentación y comunicación de las diferentes partes que conforman una aplicación antes de su implementación.

UML proporciona un conjunto de diagramas y elementos gráficos estandarizados que permiten describir componentes como clases, objetos, relaciones, procesos e interacciones. Gracias a esta representación visual, los equipos de desarrollo pueden comprender con mayor facilidad la organización del sistema, validar el diseño y realizar ajustes antes de iniciar la etapa de codificación.

Es importante destacar que UML no es un lenguaje de programación ni una metodología de desarrollo. Se trata de un lenguaje de modelado que complementa el proceso de desarrollo de software, proporcionando una notación común para representar diferentes perspectivas de una aplicación.

Entre las principales ventajas del uso de UML se encuentran:

- Facilitar la comprensión de la estructura y el funcionamiento del software.
- Favorecer la comunicación entre los integrantes del equipo de desarrollo y demás interesados en el proyecto.
- Documentar las características del sistema de manera organizada y estandarizada.
- Identificar posibles inconsistencias durante las etapas de análisis y diseño.
- Servir como guía para la implementación, el mantenimiento y la evolución del software.

La utilización de UML favorece la representación clara y organizada de un sistema de software, proporcionando una base común para su análisis, diseño, documentación y mantenimiento.

2.2. Elementos básicos del UML

UML emplea un conjunto de elementos gráficos que permiten representar la estructura y el comportamiento de un sistema de software. En el modelado orientado a objetos, estos elementos facilitan la descripción de las entidades que conforman una aplicación, sus características y la forma en que interactúan entre sí.

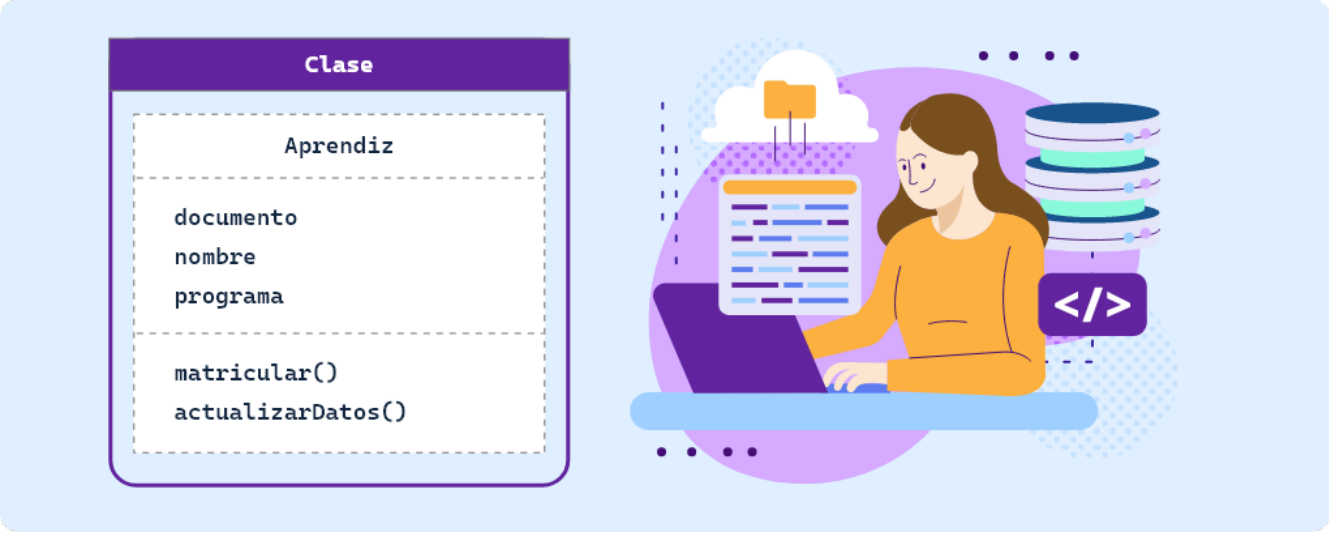
Los elementos básicos de UML constituyen la base para representar de forma gráfica la estructura de un sistema de software, facilitando su comprensión, organización y documentación durante las etapas de análisis y diseño.

Los principales elementos son los siguientes:

- **Clase:** representa una plantilla o modelo que define las características y los comportamientos comunes de un conjunto de objetos. En UML, una clase se representa mediante un rectángulo dividido en compartimentos donde se registra su nombre, atributos y operaciones.
- **Objeto:** es una instancia creada a partir de una clase durante la ejecución de una aplicación. Cada objeto posee valores propios para sus atributos y puede ejecutar los métodos definidos por la clase de la que proviene.
- **Atributos:** corresponden a las características o propiedades que describen el estado de una clase u objeto. Permiten almacenar la información necesaria para representar sus condiciones o datos.
- **Operaciones (métodos):** representan las acciones o funciones que una clase puede ejecutar. Definen el comportamiento de los objetos y permiten modificar o consultar la información almacenada en sus atributos.

- **Relaciones:** permiten representar los vínculos existentes entre dos o más clases, indicando cómo interactúan o dependen unas de otras dentro del sistema. Estas relaciones facilitan la organización y comprensión de la estructura del software.

Figura 4. Elementos básicos de UML



El conocimiento de estos elementos facilita la comprensión de los modelos de software y su representación mediante UML, favoreciendo el análisis, el diseño y la documentación de los sistemas de manera organizada y estandarizada.

2.3. Diagramas de clases

El diagrama de clases es uno de los principales diagramas de UML para representar la estructura estática de un sistema de software orientado a objetos. Mediante este diagrama es posible identificar las clases que conforman el sistema, sus

atributos, operaciones y las relaciones que existen entre ellas, permitiendo visualizar de manera organizada la estructura del sistema que se desea desarrollar.

Su elaboración facilita el análisis y diseño del software, ya que permite representar gráficamente la organización del sistema antes de iniciar la etapa de implementación. Además, constituye una herramienta útil para documentar la estructura de una aplicación y favorecer la comunicación entre los integrantes del equipo de desarrollo.

En un diagrama de clases, cada clase se representa mediante un rectángulo dividido en tres compartimentos:

- **Nombre de la clase:** identifica la clase dentro del modelo.
- **Atributos:** describen las características o propiedades de la clase.
- **Operaciones (métodos):** representan las acciones o funciones que la clase puede realizar.

Figura 5. Representación de una clase en un diagrama UML



La representación gráfica de las clases mediante UML facilita la organización y comprensión de la estructura de un sistema de software, constituyendo un recurso fundamental para el análisis, el diseño y la documentación de aplicaciones orientadas a objetos.

Además de la representación de las clases, UML emplea símbolos para indicar el nivel de visibilidad o acceso de los atributos y las operaciones. Estos símbolos corresponden a los especificadores de acceso que posteriormente se implementan mediante la sintaxis del lenguaje de programación utilizado. Esta notación facilita la interpretación de los diagramas y permite identificar el alcance de cada elemento dentro del modelo tal como se indica a continuación:

Tabla 2. Notación de visibilidad (especificadores de acceso)

Símbolo	Visibilidad	Descripción
+	Pública (public)	Puede ser utilizada desde cualquier clase del sistema.
-	Privada (private)	Solo puede utilizarse dentro de la misma clase.
#	Protegida (protected)	Puede utilizarse dentro de la clase y por las clases derivadas.
~	Paquete (package)	Puede utilizarse por las clases que pertenecen al mismo paquete.

La utilización de estas convenciones permite interpretar con mayor precisión los diagramas de clases y comprender el nivel de acceso definido para los diferentes elementos que conforman el modelo.

2.4. Relaciones entre clases

Las relaciones entre clases permiten representar la forma en que los diferentes elementos de un sistema de software se vinculan e interactúan dentro de un modelo orientado a objetos. Estas relaciones facilitan la comprensión de la estructura del sistema y muestran cómo las clases colaboran para cumplir una determinada funcionalidad.

En UML, las relaciones se representan mediante líneas y símbolos específicos que indican el tipo de vínculo existente entre las clases. Su correcta utilización favorece la organización del diseño, mejora la comunicación entre los integrantes del equipo de desarrollo y contribuye a la elaboración de modelos más claros y consistentes.

Los principales tipos de relaciones utilizados en los diagramas de clases son los siguientes:

- **Asociación:** representa un vínculo estructural entre dos o más clases, indicando que existe una relación entre ellas para intercambiar información o colaborar en el cumplimiento de una función.
- **Agregación:** es una forma especial de asociación que expresa una relación de tipo todo-parte, donde los objetos pueden existir de manera independiente del objeto que los agrupa.
- **Composición:** también representa una relación todo-parte, pero con una dependencia más fuerte. En este caso, los objetos que conforman la composición dependen del ciclo de vida del objeto principal.
- **Herencia (generalización):** representa la relación mediante la cual una clase puede heredar atributos y operaciones de otra clase, favoreciendo la reutilización de código y la organización jerárquica del modelo.

Las relaciones entre clases permiten representar de manera organizada la interacción y dependencia entre los diferentes elementos de un sistema, facilitando el análisis, el diseño y la construcción de modelos orientados a objetos con una estructura clara y coherente. Por lo tanto, a continuación, se presentan los principales tipos de relaciones entre clases en UML, junto con su símbolo y característica principal:

Tabla 3. Tipos de relaciones entre clases en UML

Relación	Símbolo UML	Característica principal
Asociación	Línea continua.	Representa un vínculo entre dos clases.
Agregación	Línea continua con rombo blanco.	Relación todo-parte con independencia entre los objetos.
Composición	Línea continua con rombo negro.	Relación todo-parte con dependencia del ciclo de vida.
Herencia (generalización)	Línea continua con flecha de triángulo vacío.	Una clase hereda atributos y operaciones de otra.

Las relaciones entre clases permiten representar de manera organizada la interacción y dependencia entre los diferentes elementos de un sistema, facilitando el análisis, el diseño y la construcción de modelos orientados a objetos con una estructura clara y coherente.

Ejemplos de relaciones entre clases:

Asociación: Aprendiz — Curso

Agregación: Curso ◇— Módulo

Composición: Formulario ◆— Campo

Herencia: Usuario △— Administrador

2.5. Herramientas CASE para el modelado de software

Las herramientas CASE (Computer-Aided Software Engineering) son aplicaciones informáticas que apoyan las diferentes actividades relacionadas con el análisis, diseño, desarrollo y documentación del software. En el modelado orientado a objetos, estas herramientas facilitan la elaboración de diagramas UML mediante interfaces gráficas que permiten representar de forma organizada la estructura y el comportamiento de un sistema.

El uso de herramientas CASE contribuye a mejorar la productividad durante el proceso de desarrollo, favorece la estandarización de los modelos y facilita la actualización de la documentación del proyecto. Además, permiten detectar inconsistencias en los diagramas y mantener una representación organizada de los diferentes componentes del sistema.

Entre las herramientas CASE más utilizadas para el modelado de software se encuentran:

Tabla 4. Herramientas CASE para el modelado de software

Herramienta	Características principales
StarUML	Permite crear diferentes tipos de diagramas UML mediante una interfaz gráfica intuitiva y opciones de personalización.
Visual Paradigm	Facilita el modelado UML, el diseño de procesos y la documentación de proyectos de software.

Herramienta	Características principales
Enterprise Architect	Ofrece funciones para el modelado, análisis y documentación de sistemas de software de diferentes niveles de complejidad.
Astah UML	Proporciona herramientas para elaborar diagramas UML y favorecer la organización del diseño de aplicaciones.

La selección de una herramienta CASE depende de las necesidades del proyecto, las funcionalidades requeridas y las características del entorno de desarrollo. Independientemente de la herramienta utilizada, todas contribuyen a representar y documentar los modelos de software de manera organizada, facilitando el trabajo colaborativo y la comprensión de la arquitectura del sistema.

3. Estructura e implementación de clases

La implementación de clases constituye una etapa fundamental en el desarrollo de aplicaciones orientadas a objetos, ya que permite transformar el diseño conceptual del software en estructuras programables que representan los diferentes elementos del sistema. Una clase define las características y comportamientos que comparten los objetos, favoreciendo la organización, reutilización y mantenimiento del código.

Durante este proceso se establecen aspectos como la definición de atributos, operaciones, especificadores de acceso y mecanismos para la creación de objetos, elementos que contribuyen a desarrollar aplicaciones con una estructura clara, organizada y acorde con los principios de la programación orientada a objetos.

3.1. Implementación de una clase

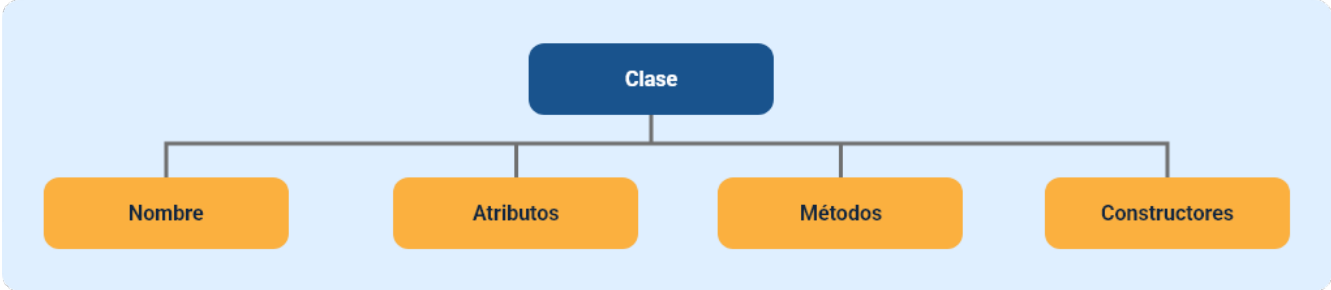
La implementación de una clase consiste en traducir el diseño realizado durante la etapa de modelado a un lenguaje de programación. En este proceso se definen los atributos que almacenan la información, las operaciones o métodos que describen el comportamiento de los objetos y la estructura que permite representar una entidad dentro del sistema.

Una clase constituye una plantilla a partir de la cual se crean objetos con características y comportamientos comunes. Su implementación debe responder a los requerimientos del sistema y mantener una organización clara que facilite la reutilización, el mantenimiento y la evolución del código.

Generalmente, una clase está conformada por los siguientes elementos:

- **Nombre de la clase:** identifica la entidad que se representa.
- **Atributos:** almacenan la información correspondiente a la clase.
- **Métodos:** definen las acciones o funcionalidades que pueden ejecutar los objetos creados a partir de la clase.
- **Constructores:** permiten inicializar los objetos al momento de su creación.
- **Especificadores de acceso:** controlan la visibilidad de los atributos y métodos dentro del programa.

Figura 6. Estructura básica de una clase



Una vez definida la estructura de la clase, se procede a implementarla mediante la sintaxis propia del lenguaje de programación seleccionado. En esta etapa se integran elementos propios del lenguaje, como los especificadores de acceso, que regulan la visibilidad de los atributos y métodos, además de otras instrucciones que permiten el correcto funcionamiento de la clase. Aunque la forma de escribir el código puede variar entre lenguajes, los principios fundamentales de la programación orientada a objetos permanecen constantes.

A continuación, se presenta un ejemplo de implementación de una clase utilizando el lenguaje de programación Java:

```
public class Aprendiz {
    private String nombre;
    private String programa;
    public Aprendiz(String nombre, String programa) {
        this.nombre = nombre;
        this.programa = programa;
    }
    public void mostrarInformacion() {
        System.out.println("Aprendiz: " + nombre);
        System.out.println("Programa: " + programa);
    }
}
```

En este ejemplo, la clase Aprendiz define dos atributos (nombre y programa), un constructor para inicializar los objetos y un método encargado de mostrar la información almacenada.

La implementación organizada de las clases favorece el desarrollo de aplicaciones más estructuradas, facilita el mantenimiento del código y promueve la reutilización de componentes dentro de un proyecto de software.

3.2. Especificadores de acceso

Como se presentó en los diagramas de clases de UML, la visibilidad de los atributos y métodos se representa mediante símbolos. Durante la implementación, estos símbolos se traducen en especificadores de acceso propios del lenguaje de programación utilizado, los cuales controlan el acceso a los diferentes elementos de la clase.

Los especificadores de acceso son palabras clave que permiten controlar la visibilidad de los atributos, métodos y otros elementos de una clase. Su principal función es establecer desde dónde pueden ser utilizados, favoreciendo la seguridad, la organización y el principio de encapsulamiento propio de la programación orientada a objetos.

La selección del especificador de acceso depende del nivel de acceso que se desea otorgar a cada elemento de la clase. De esta manera, algunos atributos o métodos pueden estar disponibles únicamente dentro de la misma clase, mientras que otros pueden ser utilizados por diferentes clases del sistema.

Los especificadores de acceso más utilizados son:

Tabla 5. Especificadores de acceso en Java

Especificador	Descripción	Uso principal
public	Permite que el atributo o método sea accesible desde cualquier clase del programa.	Funcionalidades que deben estar disponibles para otros componentes del sistema.
private	Restringe el acceso al interior de la misma clase.	Proteger la información interna y controlar su modificación.

Especificador	Descripción	Uso principal
protected	Permite el acceso desde la misma clase y desde las clases que heredan de ella.	Favorecer la reutilización mediante la herencia.
(Sin modificador)	El acceso queda restringido a las clases que pertenecen al mismo paquete.	Compartir información entre clases relacionadas sin hacerla pública.

A continuación, se presenta un ejemplo del uso de especificadores de acceso en el lenguaje de programación Java:

```
public class Aprendiz {
    // Atributo accesible solo dentro de la clase
    private String nombre;
    // Atributo accesible desde clases del mismo paquete
    String programa;
    // Atributo accesible desde clases relacionadas mediante herencia
    protected String ficha;
    // Método accesible desde cualquier clase
    public void mostrarInformacion() {
        System.out.println(nombre + " - " + programa);
    }
}
```

En el ejemplo anterior, el atributo nombre solo puede utilizarse dentro de la clase Aprendiz, mientras que programa está disponible para las clases del mismo paquete y ficha puede ser utilizada por las clases derivadas. Por su parte, el método mostrarInformacion() es público, por lo que puede ser invocado desde cualquier clase que tenga acceso al objeto.

El uso adecuado de los especificadores de acceso contribuye a proteger la información, controlar la interacción entre los diferentes componentes del sistema y desarrollar aplicaciones más organizadas, seguras y fáciles de mantener.

3.3. Constructores y administración de memoria en Java

Los constructores son métodos especiales que se ejecutan automáticamente al crear un objeto de una clase. Su función principal es inicializar los atributos del objeto, asignándoles valores iniciales y preparando la instancia para su utilización dentro de la aplicación.

Una clase puede disponer de uno o varios constructores, dependiendo de las necesidades del programa. Cuando no se define un constructor, algunos lenguajes de programación generan uno por defecto; sin embargo, es recomendable crear constructores que permitan inicializar los objetos con la información requerida.

A continuación, se presenta un ejemplo de un constructor en el lenguaje de programación Java:

```
public class Aprendiz {
    private String nombre;
    private String programa;
    public Aprendiz(String nombre, String programa) {
        this.nombre = nombre;
        this.programa = programa;
    }
    public void mostrarInformacion() {
        System.out.println(nombre + " - " + programa);
    }
}
```

En el ejemplo anterior, el constructor Aprendiz() recibe los valores correspondientes al nombre y al programa de formación, asignándolos a los atributos del objeto durante su creación. De esta manera, cada objeto inicia su ciclo de vida con la información necesaria para su funcionamiento.

En algunos lenguajes de programación también existen los destructores, cuya función es liberar los recursos utilizados por un objeto cuando este deja de ser necesario. No obstante, en Java esta tarea es administrada automáticamente por el Garbage Collector (recolector de basura), un mecanismo que identifica y libera de forma automática la memoria ocupada por objetos que ya no son utilizados por la aplicación.

Figura 7. Ciclo de vida básico de un objeto en Java



La utilización de constructores permite crear objetos correctamente inicializados y contribuye a desarrollar aplicaciones más organizadas. Asimismo, la administración automática de memoria en Java simplifica la gestión de los recursos durante la ejecución del programa y favorece un desarrollo más eficiente y seguro.

4. Herencia en la programación orientada a objetos

La herencia es uno de los principios fundamentales de la programación orientada a objetos que permite crear nuevas clases a partir de otras ya existentes. Este mecanismo favorece la reutilización del código, facilita la organización de las aplicaciones y contribuye a desarrollar soluciones más flexibles y fáciles de mantener.

Mediante la herencia es posible establecer relaciones entre clases, compartiendo atributos y métodos comunes, al tiempo que se incorporan nuevas características o comportamientos según las necesidades del sistema. Este enfoque reduce la duplicidad de código y promueve una estructura más organizada durante el desarrollo de aplicaciones.

La comprensión de este principio constituye un paso importante para diseñar aplicaciones escalables, fortalecer la reutilización de componentes y aplicar de manera eficiente los fundamentos de la programación orientada a objetos.

4.1. Principios y beneficios de la herencia

La herencia es un mecanismo de la programación orientada a objetos que permite crear una nueva clase a partir de otra ya existente. La clase que transmite sus características se denomina clase base o clase padre, mientras que la nueva clase recibe el nombre de clase derivada o clase hija. Mediante este mecanismo, la clase derivada puede reutilizar los atributos y métodos de la clase base, además de incorporar nuevas funcionalidades o modificar comportamientos según las necesidades de la aplicación.

Este principio favorece la organización del software al establecer relaciones entre clases que comparten características comunes. En lugar de duplicar código, es

posible reutilizar los elementos ya definidos en una clase existente, facilitando el mantenimiento y la evolución del sistema.

La herencia se fundamenta en el principio de especialización, mediante el cual una clase derivada conserva las características generales de la clase base y añade propiedades o comportamientos específicos. Esta organización permite representar de manera más precisa las relaciones existentes entre los diferentes elementos que conforman una aplicación.

Entre los principales beneficios de la herencia se encuentran:

- Reutilizar atributos y métodos definidos en otras clases.
- Reducir la duplicidad de código durante el desarrollo.
- Facilitar el mantenimiento y la actualización de las aplicaciones.
- Favorecer la organización jerárquica de las clases.
- Incrementar la reutilización y escalabilidad de los componentes de software.

A continuación, se presenta un ejemplo de herencia utilizando el lenguaje de programación Java:

```
class Persona {
   String nombre;
   String documento;
}
class Aprendiz extends Persona {
```

```
    String programa;
}
```

En el ejemplo anterior, la clase Aprendiz hereda los atributos nombre y documento definidos en la clase Persona, e incorpora el atributo programa, propio del contexto de formación. Gracias a la herencia, no es necesario volver a declarar los atributos heredados, lo que facilita la reutilización del código y mejora la organización de la aplicación.

La aplicación de la herencia permite desarrollar software más organizado, reutilizar componentes existentes y facilitar la incorporación de nuevas funcionalidades sin modificar la estructura de las clases ya definidas.

4.2. Herencia simple y múltiple

La herencia puede clasificarse según la forma en que una clase deriva sus características de otras clases. Los tipos más conocidos son la herencia simple y la herencia múltiple, cada uno con características y aplicaciones particulares.

La herencia simple ocurre cuando una clase deriva directamente de una única clase base. Este modelo favorece una estructura clara y organizada, facilitando la reutilización de atributos y métodos sin incrementar la complejidad del diseño. Java implementa este tipo de herencia mediante la palabra reservada extends.

A continuación, se presenta un ejemplo de herencia simple utilizando el lenguaje de programación Java:

```
class Persona {
    String nombre;
}
class Aprendiz extends Persona {
    String programa;
}
```

En este ejemplo, la clase Aprendiz hereda los atributos de la clase Persona y añade un nuevo atributo denominado programa, propio del contexto de formación.

La herencia múltiple se presenta cuando una clase puede heredar características de dos o más clases base. Aunque este mecanismo está disponible en algunos lenguajes de programación, Java no permite la herencia múltiple entre clases debido a la posibilidad de generar ambigüedades durante la ejecución del programa. No obstante, Java ofrece mecanismos alternativos, como las interfaces, que permiten compartir comportamientos entre diferentes clases sin recurrir a la herencia múltiple de clases.

A continuación, se presenta una comparación entre la herencia simple y la herencia múltiple, considerando sus principales características y su implementación en Java:

Tabla 6. Comparación entre herencia simple y herencia múltiple

Característica	Herencia simple	Herencia múltiple
Número de clases base	Una	Dos o más
Complejidad del diseño	Menor	Mayor

Característica	Herencia simple	Herencia múltiple
Disponible en Java	Sí	No (entre clases)
Alternativa en Java	No aplica	Interfaces

La comprensión de estos tipos de herencia permite seleccionar la estrategia más adecuada durante el diseño de aplicaciones orientadas a objetos, favoreciendo soluciones organizadas, reutilizables y acordes con las características del lenguaje de programación empleado.

4.3. Sobrescritura de métodos (override)

La sobrescritura de métodos es un mecanismo de la programación orientada a objetos que permite a una clase derivada proporcionar una implementación propia de un método heredado de la clase base. De esta manera, es posible adaptar o ampliar el comportamiento de un método sin modificar la definición original de la clase padre.

En Java, la sobrescritura se realiza definiendo en la clase derivada un método con el mismo nombre, los mismos parámetros y el mismo tipo de retorno que el método declarado en la clase base. Para indicar que se trata de una sobrescritura, es recomendable utilizar la anotación @Override, la cual facilita la identificación del método y permite al compilador verificar que la sobrescritura se haya realizado correctamente.

A continuación, se presenta un ejemplo de sobrescritura de métodos utilizando el lenguaje de programación Java:

```
class Persona {
    public void presentarse() {
        System.out.println("Soy una persona.");
    }
}
class Aprendiz extends Persona {
    @Override
    public void presentarse() {
        System.out.println("Soy un aprendiz del SENA.");
    }
}
```

En el ejemplo anterior, la clase Aprendiz hereda el método presentarse() de la clase Persona, pero redefine su comportamiento para expresar un mensaje diferente. Gracias a la sobrescritura, el método mantiene el mismo propósito, aunque su ejecución se adapta a las características de la clase derivada.

La sobrescritura de métodos favorece la reutilización del código y permite especializar el comportamiento de las clases derivadas, contribuyendo al desarrollo de aplicaciones más flexibles y acordes con los principios de la programación orientada a objetos.

4.4. Clases y métodos abstractos

Las clases abstractas son clases que sirven como modelo para otras clases, permitiendo definir atributos y métodos comunes que podrán ser reutilizados por las clases derivadas. Estas clases no pueden instanciarse directamente, ya que su propósito es establecer una estructura general que posteriormente será especializada mediante la herencia.

Además de contener atributos y métodos implementados, una clase abstracta puede declarar métodos abstractos, los cuales definen únicamente su nombre, parámetros y tipo de retorno, sin incluir una implementación. Las clases derivadas que heredan de una clase abstracta deben proporcionar la implementación de estos métodos, adaptándolos a sus propias necesidades.

En Java, las clases abstractas se declaran mediante la palabra reservada abstract, la cual también se utiliza para identificar los métodos abstractos.

A continuación, se presenta un ejemplo de una clase abstracta utilizando el lenguaje de programación Java:

```
abstract class Persona {
    String nombre;
    public abstract void presentarse();
}
class Aprendiz extends Persona {
    @Override
```

```
    public void presentarse() {
        System.out.println("Soy un aprendiz del SENA.");
    }
}
```

En el ejemplo anterior, la clase Persona define el atributo nombre y declara el método abstracto presentarse(), cuya implementación se realiza en la clase Aprendiz. De esta manera, todas las clases que hereden de Persona deberán proporcionar su propia implementación del método, conservando una estructura común y adaptando su comportamiento según el contexto.

El uso de clases y métodos abstractos favorece la reutilización del código, establece una estructura común para las clases relacionadas y facilita el desarrollo de aplicaciones organizadas, flexibles y acordes con los principios de la programación orientada a objetos.

4.5. Constructores en la herencia

En una relación de herencia, las clases derivadas también utilizan constructores para inicializar sus propios atributos. Cuando se crea un objeto de una clase derivada, primero se ejecuta el constructor de la clase base y, posteriormente, el constructor de la clase derivada. Este proceso garantiza que los atributos heredados y los propios de la nueva clase queden correctamente inicializados.

En Java, la invocación del constructor de la clase base se realiza mediante la palabra reservada super, la cual permite acceder al constructor de la clase padre y

reutilizar su proceso de inicialización. De esta manera, se evita duplicar código y se mantiene la coherencia durante la creación de los objetos.

A continuación, se presenta un ejemplo del uso de constructores en una relación de herencia utilizando el lenguaje de programación Java:

```
class Persona {
    String nombre;
    public Persona(String nombre) {
        this.nombre = nombre;
    }
}
class Aprendiz extends Persona {
    String programa;
    public Aprendiz(String nombre, String programa) {
        super(nombre);
        this.programa = programa;
    }
}
```

En el ejemplo anterior, el constructor de la clase Aprendiz invoca mediante super el constructor de la clase Persona para inicializar el atributo nombre. Posteriormente, inicializa el atributo programa, propio de la clase derivada. Este procedimiento

garantiza que el objeto se cree con toda la información necesaria, respetando la relación de herencia entre ambas clases.

La utilización de constructores en la herencia favorece la reutilización del código, mantiene una adecuada organización de las clases y asegura una correcta inicialización de los objetos durante el desarrollo de aplicaciones orientadas a objetos.

5. Polimorfismo y reutilización de código

El polimorfismo es uno de los principios fundamentales de la programación orientada a objetos y complementa conceptos como la encapsulación, la herencia y la abstracción. Su principal característica consiste en permitir que un mismo método o una misma referencia pueda adoptar diferentes comportamientos según el objeto con el que interactúe, favoreciendo el desarrollo de aplicaciones más flexibles y adaptables.

Gracias al polimorfismo, es posible diseñar soluciones que reutilizan estructuras comunes y, al mismo tiempo, permiten que cada clase implemente comportamientos específicos de acuerdo con sus necesidades. Este principio facilita la organización del código, reduce la duplicidad y contribuye al mantenimiento y evolución de las aplicaciones, especialmente cuando se desarrollan proyectos de mayor complejidad.

La incorporación del polimorfismo en el desarrollo de software contribuye a mejorar la organización del código, facilita su mantenimiento y favorece la construcción de aplicaciones más flexibles mediante la reutilización de estructuras y comportamientos comunes.

5.1. Principios y beneficios del polimorfismo

El polimorfismo es uno de los principios fundamentales de la programación orientada a objetos que permite que un mismo método o una misma referencia presente diferentes comportamientos según el objeto sobre el cual actúe. Esta característica favorece la reutilización del código y proporciona mayor flexibilidad en el desarrollo de aplicaciones, ya que una misma operación puede adaptarse a distintas clases relacionadas mediante la herencia.

El funcionamiento del polimorfismo se basa en la posibilidad de utilizar una referencia de una clase base para acceder a objetos de sus clases derivadas. Aunque los objetos comparten una estructura común, cada uno puede ejecutar una implementación diferente de un mismo método, de acuerdo con las características definidas en su propia clase.

En Java, el polimorfismo se manifiesta principalmente mediante la herencia y la sobrescritura de métodos. Esto permite que una referencia de la clase padre invoque el comportamiento correspondiente al objeto que representa en tiempo de ejecución, favoreciendo la construcción de aplicaciones más dinámicas, flexibles y fáciles de mantener.

Entre los principales beneficios del polimorfismo se destacan:

- Facilita el mantenimiento y la evolución de las aplicaciones al incorporar nuevas clases sin modificar el código existente.
- Favorece la reutilización del código mediante el uso de estructuras comunes.
- Reduce la duplicidad de código al permitir que varias clases compartan una misma interfaz o comportamiento general.
- Promueve el desarrollo de aplicaciones más flexibles y escalables.
- Mejora la organización del software al separar los comportamientos generales de las implementaciones específicas.

A continuación, se presenta un ejemplo conceptual del uso del polimorfismo en el lenguaje de programación Java:

```
class Persona {
    public void presentarse() {
        System.out.println("Soy una persona.");
    }
}
class Aprendiz extends Persona {
    @Override
    public void presentarse() {
        System.out.println("Soy un aprendiz del SENA.");
    }
}
class Instructor extends Persona {
    @Override
    public void presentarse() {
        System.out.println("Soy un instructor del SENA.");
    }
}
public class Principal {
```

```
    public static void main(String[] args) {
        Persona persona1 = new Aprendiz();
        Persona persona2 = new Instructor();
        persona1.presentarse();
        persona2.presentarse();
    }
}
```

En el ejemplo anterior, las variables persona1 y persona2 son referencias de la clase Persona, pero cada una apunta a un objeto de una clase diferente. Al invocar el método presentarse(), se ejecuta la implementación correspondiente al tipo real del objeto, evidenciando el comportamiento polimórfico.

El polimorfismo constituye un principio esencial para el desarrollo de aplicaciones orientadas a objetos, ya que facilita la reutilización del código, mejora la organización de las clases y permite extender la funcionalidad del software de manera flexible y mantenible.

5.2. Sobrecarga de métodos

La sobrecarga de métodos es una característica de la programación orientada a objetos que permite definir varios métodos con el mismo nombre dentro de una misma clase, siempre que se diferencien por la cantidad, el tipo o el orden de sus parámetros. Este mecanismo facilita la reutilización de nombres de métodos para realizar

operaciones relacionadas, adaptándolas a diferentes necesidades sin generar conflictos durante la compilación.

En Java, el compilador identifica cuál método debe ejecutarse de acuerdo con los argumentos enviados en la llamada. De esta manera, es posible utilizar un mismo nombre para métodos que realizan una función similar, pero que reciben información diferente.

La siguiente tabla presenta las principales características de la sobrecarga de métodos:

Tabla 7. Características de la sobrecarga de métodos en Java

Característica	Descripción
Nombre del método	Permanece igual en todas las versiones del método.
Parámetros	Deben variar en cantidad, tipo u orden.
Tipo de retorno	Puede ser igual o diferente, pero no determina la sobrecarga por sí solo.
Clase	Los métodos pertenecen a la misma clase.
Momento de resolución	El compilador determina qué método ejecutar según los argumentos recibidos.

A continuación, se presenta un ejemplo de sobrecarga de métodos utilizando el lenguaje de programación Java:

```
class Calculadora {
    public int sumar(int a, int b) {
        return a + b;
```

```
    }

    public int sumar(int a, int b, int c) {

        return a + b + c;

    }

    public double sumar(double a, double b) {

        return a + b;

    }

}
```

En el ejemplo anterior, el método sumar() se encuentra definido tres veces dentro de la misma clase. Aunque conserva el mismo nombre, cada versión recibe parámetros diferentes, lo que permite al compilador identificar automáticamente cuál implementación debe ejecutarse según los argumentos suministrados.

La sobrecarga de métodos contribuye a mejorar la legibilidad del código, favorece la reutilización de funcionalidades relacionadas y simplifica el desarrollo de aplicaciones al ofrecer diferentes alternativas para realizar una misma operación.

5.3. Sobrescritura y polimorfismo

La sobrescritura de métodos constituye uno de los mecanismos que hacen posible el polimorfismo en la programación orientada a objetos. Mediante este proceso, una clase derivada redefine el comportamiento de un método heredado de la

clase base, permitiendo que cada objeto responda de manera diferente ante una misma operación.

En Java, cuando una referencia de la clase padre apunta a un objeto de una clase derivada, la ejecución del método sobrescrito depende del tipo real del objeto y no del tipo de la referencia. Este comportamiento se determina durante la ejecución del programa, lo que permite desarrollar aplicaciones más flexibles y adaptables a diferentes escenarios.

En este punto es importante identificar las principales diferencias entre la sobrecarga y la sobrescritura de métodos.

A continuación, se presentan las principales diferencias entre la sobrecarga y la sobrescritura de métodos, considerando aspectos como su ubicación, parámetros, relación con la herencia, momento de resolución y propósito:

Tabla 8. Comparación entre sobrecarga y sobrescritura de métodos

Característica	Sobrecarga	Sobrescritura
Ubicación	Misma clase	Clase derivada
Nombre del método	Igual	Igual
Parámetros	Diferentes	Iguales
Herencia	No es necesaria	Requiere herencia
Momento de resolución	En compilación	En ejecución
Propósito	Ofrecer diferentes versiones de un método	Modificar el comportamiento heredado

Posteriormente, se presenta un ejemplo de sobrescritura y polimorfismo utilizando el lenguaje de programación Java:

```
class Persona {
    public void presentarse() {
        System.out.println("Soy una persona.");
    }
}
class Aprendiz extends Persona {
    @Override
    public void presentarse() {
        System.out.println("Soy un aprendiz del SENA.");
    }
}
class Instructor extends Persona {
    @Override
    public void presentarse() {
        System.out.println("Soy un instructor del SENA.");
    }
}
```

```
public class Principal {

    public static void main(String[] args) {


        Persona persona;

        persona = new Aprendiz();

        persona.presentarse();

        persona = new Instructor();

        persona.presentarse();

    }

}
```

En el ejemplo anterior, la variable persona corresponde a una referencia de la clase Persona, pero durante la ejecución puede representar objetos de diferentes clases derivadas. Aunque la llamada al método presentarse() es la misma, el resultado cambia según el tipo de objeto asignado, evidenciando el comportamiento polimórfico.

La combinación de la herencia y la sobrescritura permite desarrollar aplicaciones más flexibles y reutilizables, ya que facilita la incorporación de nuevas clases con comportamientos específicos sin modificar la estructura general del programa.

5.4. Aplicación del polimorfismo mediante herencia

La aplicación del polimorfismo mediante herencia permite desarrollar aplicaciones en las que una misma referencia puede gestionar objetos pertenecientes a diferentes clases derivadas. Este mecanismo facilita la reutilización del código y posibilita la incorporación de nuevas funcionalidades sin modificar la lógica general de la aplicación.

En Java, una referencia de la clase base puede almacenar objetos de cualquiera de sus clases derivadas. Gracias a ello, es posible invocar un mismo método sobre distintos objetos y obtener comportamientos específicos según la implementación realizada en cada clase. Esta característica favorece la extensibilidad del software y simplifica la administración de las clases relacionadas.

A continuación, se presenta un ejemplo de aplicación del polimorfismo mediante herencia utilizando el lenguaje de programación Java:

```
class Persona {
    public void realizarActividad() {
        System.out.println("Realizando una actividad.");
    }
}
class Aprendiz extends Persona {
    @Override
```

```
    public void realizarActividad() {
        System.out.println("El aprendiz desarrolla una actividad de formación.");
    }
}
class Instructor extends Persona {
    @Override
    public void realizarActividad() {
        System.out.println("El instructor orienta el proceso de formación.");
    }
}
public class Principal {
    public static void main(String[] args) {
        Persona[] personas = {
            new Aprendiz(),
            new Instructor()
        };
        for (Persona persona : personas) {
            persona.realizarActividad();
        }
```

```
    }
}
```

En el ejemplo anterior, el arreglo personas almacena objetos de las clases Aprendiz e Instructor utilizando referencias de tipo Persona. Durante el recorrido del arreglo, la llamada al método realizarActividad() produce un comportamiento diferente para cada objeto, demostrando cómo el polimorfismo permite tratar distintos tipos de objetos de manera uniforme sin perder sus características particulares.

La aplicación del polimorfismo mediante herencia constituye una práctica ampliamente utilizada en el desarrollo de software, ya que facilita la reutilización del código, mejora la organización de las aplicaciones y permite incorporar nuevas clases con un impacto mínimo sobre la estructura existente.

6. Desarrollo y depuración de aplicaciones

El desarrollo de aplicaciones requiere un proceso organizado que inicia con el análisis del problema y el diseño de una solución lógica antes de su implementación en un lenguaje de programación. Una planificación adecuada de los algoritmos facilita la construcción de programas más claros, eficientes y acordes con los requerimientos establecidos.

Durante la implementación de una aplicación, es común que se presenten errores relacionados con la sintaxis, la lógica o el funcionamiento del programa. Por esta razón, la depuración y las pruebas constituyen actividades fundamentales para identificar, analizar y corregir estas situaciones, garantizando que el software produzca los resultados esperados.

La aplicación de estrategias de diseño, depuración y validación contribuye al desarrollo de programas más confiables, facilita su mantenimiento y fortalece la calidad de las soluciones de software implementadas.

6.1. Diseño e implementación de algoritmos

El desarrollo de una aplicación inicia con el diseño de un algoritmo que describa de manera ordenada los pasos necesarios para resolver un problema. Un algoritmo bien estructurado facilita la comprensión de la solución, reduce la probabilidad de errores durante la programación y favorece el mantenimiento del código.

Durante la etapa de diseño es posible emplear herramientas como el pseudocódigo o los diagramas de flujo para representar la lógica de la solución antes de escribir el programa. Estas representaciones permiten analizar la secuencia de

instrucciones, identificar decisiones y validar el funcionamiento general del algoritmo antes de su implementación.

Una vez definida la solución, el algoritmo se implementa utilizando la sintaxis del lenguaje de programación seleccionado. En esta etapa, cada instrucción lógica se traduce en código fuente, incorporando estructuras de control, variables, métodos y demás elementos necesarios para construir una aplicación funcional.

Las principales etapas del proceso de diseño e implementación de un algoritmo son:

Figura 8. Proceso de diseño e implementación de un algoritmo



A continuación, se presenta un ejemplo de implementación de un algoritmo en el lenguaje de programación Java:

```
public class MayorEdad {
    public static void main(String[] args) {
        int edad = 20;
        if (edad >= 18) {
            System.out.println("Es mayor de edad.");
        } else {
            System.out.println("Es menor de edad.");
        }
    }
}
```

En el ejemplo anterior, el algoritmo resuelve una situación sencilla mediante una estructura condicional. A partir del valor asignado a la variable edad, el programa evalúa una condición y muestra el mensaje correspondiente. Este proceso evidencia cómo una solución previamente diseñada puede implementarse mediante instrucciones propias del lenguaje Java.

El diseño e implementación de algoritmos constituye una etapa fundamental en el desarrollo de software, ya que proporciona una base organizada para construir aplicaciones funcionales, facilita la identificación de posibles errores y contribuye a obtener soluciones más claras, eficientes y mantenibles.

6.2. Depuración de programas

La depuración de programas es el proceso mediante el cual se identifican, analizan y corrigen los errores presentes en una aplicación durante su desarrollo. Esta actividad permite verificar que el programa funcione de acuerdo con los requisitos establecidos y produzca los resultados esperados antes de su implementación o puesta en funcionamiento.

Durante la depuración, el desarrollador examina el comportamiento del programa para localizar la causa de un error y determinar la mejor alternativa para solucionarlo. Este proceso puede apoyarse en herramientas integradas en los entornos de desarrollo, las cuales permiten ejecutar el programa de manera controlada, inspeccionar el contenido de las variables y realizar un seguimiento detallado de la ejecución del código.

Entre las principales acciones realizadas durante la depuración se encuentran:

- Ejecutar el programa de manera controlada.
- Identificar la ubicación donde se presenta el error.
- Revisar el contenido de las variables durante la ejecución.
- Analizar el flujo del programa para verificar su comportamiento.
- Aplicar las correcciones necesarias y comprobar nuevamente el funcionamiento de la aplicación.

Las etapas generales del proceso de depuración de un programa son:

Figura 9. Proceso general de depuración de programas



La depuración constituye una práctica esencial durante el desarrollo de software, ya que permite detectar errores de manera oportuna, mejorar la calidad del código y garantizar que la aplicación cumpla con el comportamiento esperado.

6.3. Fallas de sintaxis y fallas de lógica

Durante el desarrollo de una aplicación es frecuente encontrar errores que afectan su funcionamiento. Estos errores pueden impedir que el programa se ejecute o provocar resultados diferentes a los esperados. Identificar el tipo de falla facilita el proceso de depuración y permite aplicar la corrección más adecuada.

Las fallas más comunes durante el desarrollo de programas corresponden a errores de sintaxis y errores de lógica. Mientras las primeras están relacionadas con el incumplimiento de las reglas del lenguaje de programación, las segundas se presentan cuando el algoritmo o la implementación no producen el resultado esperado, aunque el programa pueda ejecutarse correctamente. Con el fin de facilitar su diferenciación, la siguiente tabla presenta sus principales características y algunos ejemplos representativos:

Tabla 9. Comparación entre fallas de sintaxis y fallas de lógica

Tipo de falla	Características	Ejemplo
Sintaxis	Incumple las reglas del lenguaje de programación e impide la compilación o ejecución del programa.	Olvidar un punto y coma (;), escribir incorrectamente una palabra reservada o no cerrar una llave (}).
Lógica	El programa se ejecuta, pero produce resultados incorrectos debido a un error en el algoritmo o en la implementación.	Utilizar una condición equivocada en una estructura if o realizar un cálculo con una fórmula incorrecta.

A continuación, se presentan ejemplos de fallas de sintaxis y de lógica en el lenguaje de programación Java.

Ejemplo de falla de sintaxis:

```
public class Ejemplo {
   public static void main(String[] args) {
      int edad = 20
      System.out.println(edad);
   }
}
```

En este ejemplo, la instrucción que declara la variable edad no finaliza con el punto y coma (;), por lo que el compilador genera un error de sintaxis y el programa no puede ejecutarse.

Ejemplo de falla de lógica:

```
public class Ejemplo {
    public static void main(String[] args) {
        int edad = 20;
        if (edad < 18) {
            System.out.println("Es mayor de edad.");
        } else {
            System.out.println("Es menor de edad.");
        }
    }
}
```

En este caso, el programa se ejecuta correctamente desde el punto de vista de la sintaxis; sin embargo, la condición utilizada en la estructura if es incorrecta, por lo que el resultado obtenido no corresponde con la edad evaluada. Este tipo de situación corresponde a una falla de lógica.

Reconocer la diferencia entre las fallas de sintaxis y las fallas de lógica permite optimizar el proceso de depuración, facilita la identificación de los errores y contribuye al desarrollo de aplicaciones más confiables y de mejor calidad.

6.4. Pruebas y corrección de errores

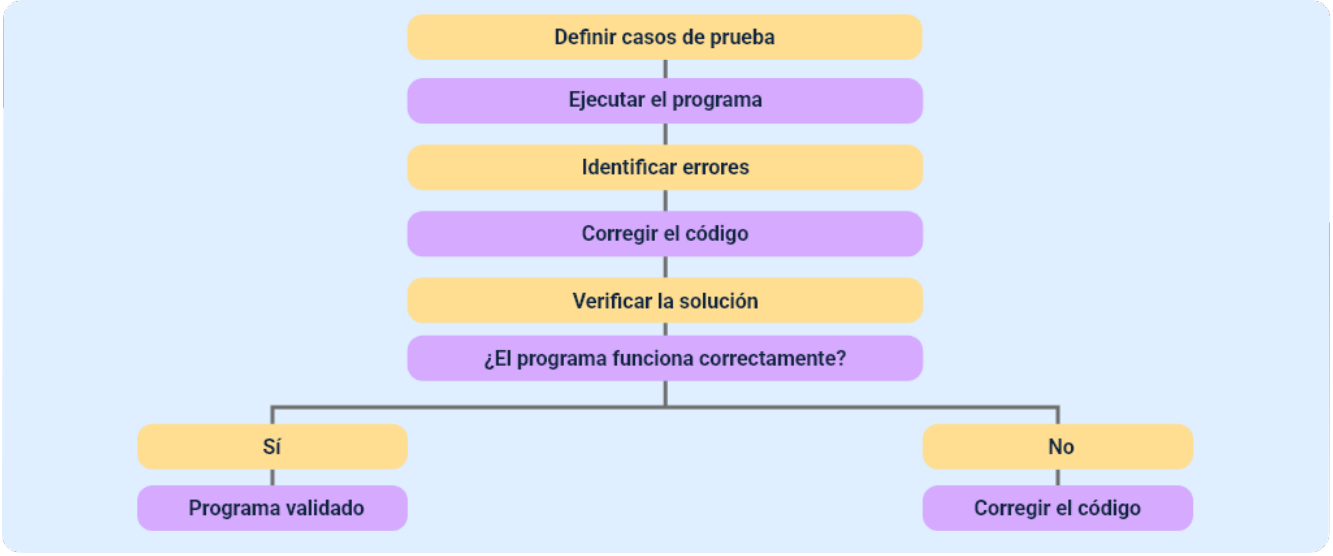
Las pruebas constituyen una actividad fundamental durante el desarrollo de software, ya que permiten comprobar que una aplicación cumple con los requisitos establecidos y produce los resultados esperados. Mediante la ejecución de diferentes casos de prueba es posible identificar errores que no fueron detectados durante la implementación y verificar el comportamiento del programa en distintas situaciones.

Una vez identificado un error, se procede a analizar su causa y realizar las modificaciones necesarias en el código fuente. Posteriormente, el programa debe ejecutarse nuevamente para comprobar que la corrección implementada solucionó el problema sin afectar otras funcionalidades de la aplicación. Este proceso puede repetirse varias veces hasta obtener un funcionamiento adecuado.

Entre las principales actividades que hacen parte del proceso de pruebas y corrección de errores se encuentran:

- Definir los casos de prueba que permitan validar el funcionamiento del programa.
- Ejecutar la aplicación con diferentes datos de entrada.
- Comparar los resultados obtenidos con los resultados esperados.
- Identificar y corregir los errores encontrados.
- Repetir las pruebas para verificar la efectividad de las correcciones realizadas.

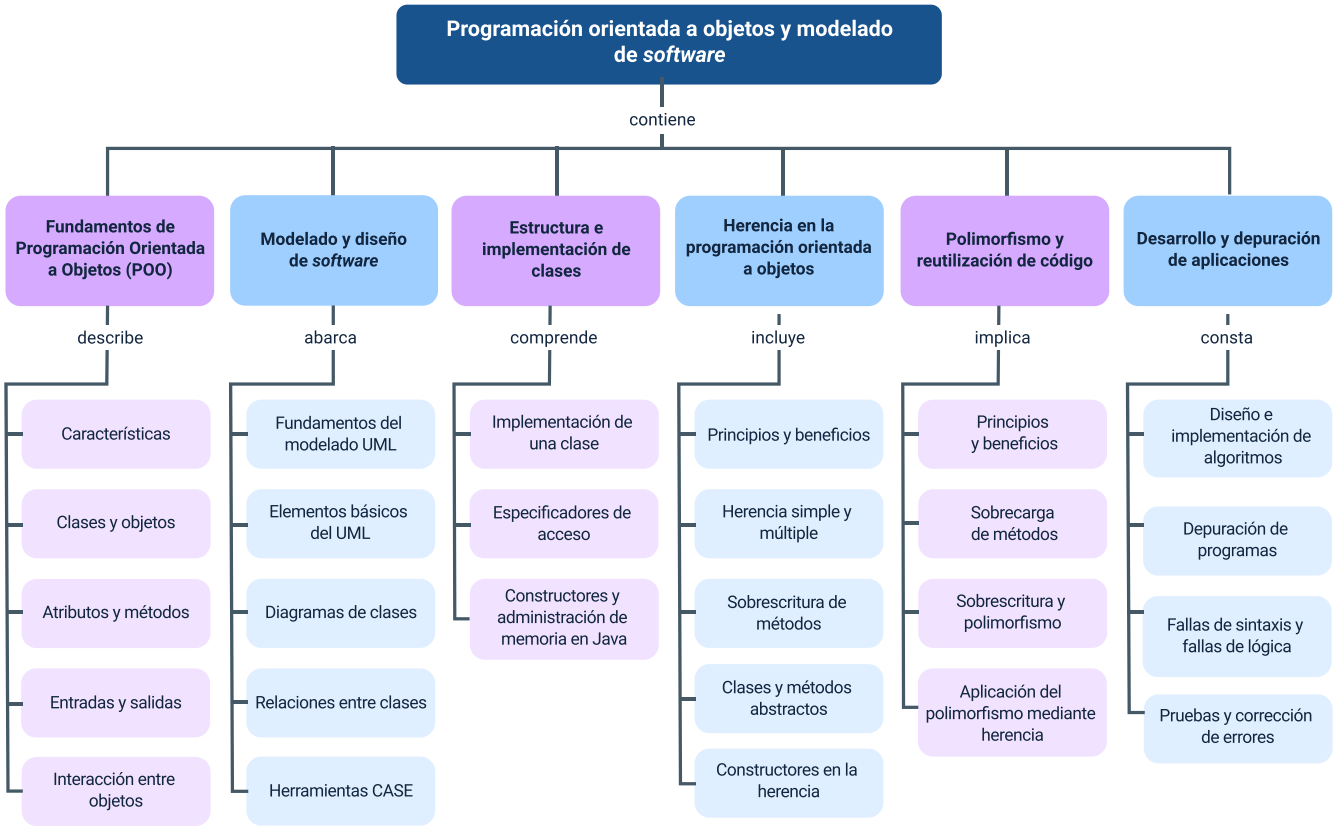
Figura 10. Ciclo de pruebas y corrección de errores



Las pruebas y la corrección de errores constituyen actividades permanentes dentro del ciclo de desarrollo de software, ya que contribuyen a mejorar la calidad de las aplicaciones, aumentar su confiabilidad y garantizar que respondan adecuadamente a las necesidades de los usuarios.

Síntesis

El componente formativo aborda los fundamentos de la programación orientada a objetos, incluyendo las características de este paradigma, la definición de clases, objetos, atributos y métodos, así como la interacción entre ellos. Además, desarrolla el modelado y diseño de software mediante UML y herramientas CASE, la implementación de clases en Java, los principios de la herencia y el polimorfismo para favorecer la reutilización del código, y las buenas prácticas para el desarrollo de aplicaciones, abarcando el diseño e implementación de algoritmos, la depuración de programas y las pruebas necesarias para identificar y corregir errores, con el propósito de construir soluciones de software organizadas, mantenibles y de calidad.



Glosario

Algoritmo: conjunto ordenado y finito de instrucciones que permiten resolver un problema o realizar una tarea específica.

Atributo: característica o dato que define el estado de un objeto dentro de una clase.

Clase: plantilla o modelo que define los atributos y métodos que compartirán los objetos creados a partir de ella.

Constructor: método especial utilizado para inicializar un objeto cuando se crea una instancia de una clase.

Depuración: proceso de identificación, análisis y corrección de errores presentes en un programa.

Diagrama de clases: representación gráfica de las clases, sus atributos, métodos y relaciones, utilizada en el modelado UML.

Encapsulamiento: principio de la programación orientada a objetos que protege los datos de un objeto mediante el control de acceso a sus atributos y métodos.

Herencia: mecanismo que permite a una clase adquirir atributos y métodos de otra, favoreciendo la reutilización del código.

Instancia: objeto creado a partir de una clase, con sus propios valores para los atributos definidos.

Método: conjunto de instrucciones que define una acción o comportamiento que puede realizar un objeto.

Polimorfismo: capacidad de utilizar una misma referencia para invocar diferentes comportamientos según el tipo de objeto que la implementa.

Programación Orientada a Objetos (POO): paradigma de programación basado en la organización del software mediante clases y objetos que interactúan entre sí.

Sobrecarga de métodos: técnica que permite definir varios métodos con el mismo nombre, pero con diferentes listas de parámetros dentro de una misma clase.

Sobrescritura de métodos: mecanismo mediante el cual una clase derivada redefine el comportamiento de un método heredado de la clase base.

UML (Lenguaje Unificado de Modelado): lenguaje estandarizado que permite representar gráficamente la estructura y el comportamiento de sistemas de software mediante diferentes tipos de diagramas.

Referencias bibliográficas

Booch, G., Rumbaugh, J., & Jacobson, I. (2006). *El lenguaje unificado de modelado. Manual de referencia* (2.ª ed.). Pearson Educación.

Fowler, M. (2004). *UML distilled: A brief guide to the standard object modeling language (3rd ed.).* Addison-Wesley.

Larman, C. (2005). Applying UML and patterns: An introduction to object-oriented analysis and design and iterative development (3rd ed.). Prentice Hall.

Oracle. (s. f.). Java documentation. https://docs.oracle.com/en/java/

Oracle. (s. f.). *Java Language and Virtual Machine Specifications.* https://docs.oracle.com/javase/specs/

Pressman, R. S., & Maxim, B. R. (2020*). Ingeniería del software: Un enfoque práctico* (9.ª ed.). McGraw-Hill.

Schildt, H. (2022). *Java: The complete reference (13th ed.).* McGraw-Hill.

Sommerville, I. (2016). *Ingeniería de software* (10.ª ed.). Pearson Educación.

Object Management Group. (2017). *Unified Modeling Language* (OMG UML). https://www.omg.org/spec/UML/2.5.1/PDF

Créditos

Nombre	Cargo	Centro de Formación y Regional
Claudia Johanna Gómez Pérez	Profesional G06. Responsable Ecosistema Virtual de Recursos Educativos Digitales	Centro Agroturístico – Regional Santander
Diana Rocío Possos Beltrán	Responsable de línea de producción	Centro de Comercio y Servicios - Regional Tolima
Viviana Esperanza Herrera Quiñonez	Evaluadora instruccional	Centro de Comercio y Servicios - Regional Tolima
Andrés Felipe Velandia Espitia	Evaluador instruccional	Centro de Comercio y Servicios - Regional Tolima
Jose Yobani Penagos Mora	Diseñador de contenidos digitales	Centro de Comercio y Servicios - Regional Tolima
José Jaime Luis Tang Pinzón	Diseñador de contenidos digitales	Centro de Comercio y Servicios - Regional Tolima
Sebastian Trujillo Afanador	Desarrollador full stack	Centro de Comercio y Servicios - Regional Tolima
Manuel Felipe Echavarria Orozco	Desarrollador full stack	Centro de Comercio y Servicios - Regional Tolima
Gilberto Junior Rodríguez Rodríguez	Animador y productor audiovisual	Centro de Comercio y Servicios - Regional Tolima
María Fernanda Pineda Mora	Evaluadora de contenidos inclusivos y accesibles	Centro de Comercio y Servicios - Regional Tolima
Jorge Bustos Gómez	Validador y vinculador de recursos educativos digitales	Centro de Comercio y Servicios - Regional Tolima